

CHRONIX EDU

The Test

Proof that holds everything together

Version 1.0 | Chronix Technology Limited | 42 Test Specs

Golden rule	A task is not DONE until its test passes. No exceptions.
Test before next task	Run the test for task N before starting task N+1. Do not batch tests.
Test structure	GIVEN (setup) — WHEN (action) — THEN (expected result)
Pass criteria	Explicit, measurable. If it cannot be measured, it is not a criterion.
Fail criteria	What a broken implementation looks like. Know the failure before you code.
Test tool	Jest + Supertest for API tests. Playwright for UI tests. Manual for visual checks.
Blocking rule	A FAIL stops all work. Fix the failure before moving to any other task.
Total tests	42 tests — 8 Foundation, 13 Phase 1, 13 Phase 2, 8 Phase 3 stretch

FOUNDATION TESTS

Tests T1–T8 · Tasks F1–F8 · Must all pass before Phase 1 begins

TEST T1 Project scaffolds and both apps start without errors → Task F1

GIVEN	A fresh clone of the chronixedu repository with .env filled from .env.example.
WHEN	Run npm install at root, then npm run dev in /apps/web and npm run dev in /apps/api simultaneously.
THEN	Both apps start without errors. Next.js serves on localhost:3000. Express API serves on localhost:3001. No TypeScript compilation errors. Path alias @/ resolves correctly in both apps.
✓ PASS IF	Both apps start with zero errors and zero TypeScript warnings.
✗ FAIL IF	Either app fails to start, TypeScript errors exist, or @/ imports fail to resolve.
NOTE	<i>Check that tsconfig.json in both apps has paths configured and that jsconfig.json is present. Run tsc --noEmit in each app before marking pass.</i>

TEST T2 Database schema applies cleanly with all 16 tables and new fields → Task F2

GIVEN	A fresh Supabase project with no existing tables. Migration file 001_initial_schema.sql is ready.
WHEN	Run the migration file against the Supabase database.
THEN	All 16 tables created. users table has title VARCHAR(20) NULLABLE and teacher_mode ENUM('class','subject') columns. assessment_components CHECK constraint prevents weight sum != 100. Partial unique index on academic_sessions(school_id) WHERE is_current=true exists. All school_id indexes exist.
✓ PASS IF	All 16 tables exist. title and teacher_mode columns exist on users. CHECK constraint and partial unique index verified with \d+ commands.
✗ FAIL IF	Any table missing. title or teacher_mode missing from users. CHECK constraint absent. Partial unique index absent.
NOTE	<i>Run SELECT table_name FROM information_schema.tables WHERE table_schema='public' to verify all 16 tables. Run \d users to verify new columns.</i>

TEST T3 Row Level Security — School A cannot read School B data → Task F3

GIVEN	Two schools created (School A and School B). One student registered in each school. Two JWTs generated — one per school.
WHEN	Using School B's JWT, make a GET request to /api/schools/:schoolAId/students.
THEN	The response returns an empty array or a 403 Forbidden. Zero students from School A are returned. No error is thrown — the system handles it gracefully.
✓ PASS IF	Response is empty array or 403. Absolutely zero School A student records appear in the response.
✗ FAIL IF	Any School A student record appears in the response body. This is a critical security failure — stop all work immediately.

NOTE	<i>This test must pass with 100% certainty before any other task proceeds. Run it three times with different data combinations to be sure.</i>
------	--

TEST T4 JWT login returns token with correct custom claims → Task F4	
GIVEN	A user account exists with role='teacher', school_id set, and title='Mr' in the users table.
WHEN	POST /api/auth/login with valid email and password.
THEN	Response includes a JWT token. Decoding the token reveals claims: { user_id, school_id, role: 'teacher', email, title: 'Mr' }. Token expires in the configured duration. verifyToken middleware accepts the token. requireRole('teacher') passes. requireRole('principal') returns 403.
✓ PASS IF	JWT decodes with all 5 claims present including title. requireRole passes for correct role and blocks for incorrect role.
✗ FAIL IF	title claim missing from JWT. Any claim missing. requireRole allows wrong role through.
NOTE	<i>Use jwt.io or the Node jwt.verify() function to inspect decoded token claims during testing.</i>

TEST T5 School creation seeds default grading scale and assessment components → Task F5	
GIVEN	No schools exist in the database.
WHEN	POST /api/schools with valid school data (name, slug, address, phone, email, subscription_tier).
THEN	School row created. school_settings row auto-created with: identity_config containing school name, academic_config containing grading scale (A=70-100, B=60-69, C=50-59, D=40-49, F=0-39) and promotion_cutoff=40, assessment components (CA1 10%, CA2 10%, Mid-Term 10%, Exam 70%). All weights sum to 100.
✓ PASS IF	school_settings row exists for the new school. Grading scale and assessment components match the Nigerian defaults exactly. Weight sum = 100.
✗ FAIL IF	school_settings row not created. Any default value missing or incorrect. Weights do not sum to 100.
NOTE	<i>Query school_settings directly: SELECT academic_config FROM school_settings WHERE school_id = [new_id] and verify JSON structure.</i>

TEST T6 Current-context endpoint returns active session and term, cached for 60 seconds → Task F6	
GIVEN	A school exists with one active session containing one active term. A second session exists with is_current=false.
WHEN	GET /api/schools/:schoolId/current-context twice within 60 seconds.
THEN	First call returns the active session and term with correct names and dates. Second call returns identical data from cache (verify via response time — cached response < 5ms). Attempting to set two sessions as is_current=true simultaneously fails with a database constraint error.

✓ PASS IF	Correct active session and term returned. Second call is served from cache. Simultaneous activation constraint error confirmed.
✗ FAIL IF	Wrong session returned. Cache not working (both calls hit DB). Two sessions can be active simultaneously.
NOTE	Test the partial unique index by attempting: <code>UPDATE academic_sessions SET is_current=true WHERE school_id=[id]</code> — both rows should not be allowed to be true simultaneously.

TEST T7 Teacher assignment mode locked per account — UI mode determined by teacher_mode field → Task F7

GIVEN	Two teacher accounts: Teacher A with teacher_mode='class', Teacher B with teacher_mode='subject'. Both assigned to classes and subjects.
WHEN	Log in as Teacher A and navigate to score entry. Log in as Teacher B and navigate to score entry.
THEN	Teacher A sees class-first score entry (class selector first, then subject). Teacher B sees subject-first score entry (assignment overview cards, then class tabs). Neither teacher sees a mode toggle. teacher_mode cannot be changed from the teacher's own profile page.
✓ PASS IF	Teacher A gets class mode. Teacher B gets subject mode. No toggle visible to either teacher.
✗ FAIL IF	Either teacher sees the mode toggle. Wrong mode shown for either teacher. teacher_mode changeable by the teacher themselves.
NOTE	The teacher_mode field is set at user creation by the admin. It should not appear as an editable field in the teacher's own settings page.

TEST T8 Assessment config resolution returns correct config using 4-level priority → Task F8

GIVEN	Four configs exist for the same school and term: (1) subject=Mathematics + class_level=JSS, (2) subject=Mathematics only, (3) class_level=JSS only, (4) is_default=true school-wide.
WHEN	Call GET /assessment-config/resolve with four combinations: (a) class_id=JSS2A + subject_id=Mathematics, (b) class_id=SS1A + subject_id=Mathematics, (c) class_id=JSS2A + subject_id=English, (d) class_id=SS1A + subject_id=English.
THEN	(a) returns config 1 — most specific wins. (b) returns config 2 — subject match without class level. (c) returns config 3 — class level match. (d) returns config 4 — school-wide default. All four return different configs.
✓ PASS IF	All four combinations return the correct config matching the priority hierarchy.
✗ FAIL IF	Any combination returns the wrong config. All combinations return the same config. Default returned when a more specific one should match.
NOTE	This is the most complex query in the foundation. Run all four cases. If any fails, the score entry system will compute wrong grades.

PHASE 1 TESTS

Tests T9–T21 · Tasks M1–M13 · All must pass before Phase 2 begins

TEST T9 Student registration creates student, parent accounts, and admission number → Task M1

GIVEN	A school exists with an active session and class JSS2A.
WHEN	POST /api/schools/:schoolId/students with full bio data, class_id=JSS2A, and one parent (name, email, phone, relationship_type='mother', is_primary_contact=true).
THEN	Student row created with auto-generated admission_no in format SCH-2025-0001. user row created with role='student'. Parent user row created with role='parent'. parent_students row created linking parent to student. student_classes row created for current session.
✓ PASS IF	All 5 rows exist. Admission number format correct. Parent linked to student via parent_students.
✗ FAIL IF	Any row missing. Admission number in wrong format. Second student gets same admission number.
NOTE	Register a second student and verify admission number increments to SCH-2025-0002.

TEST T10 Score entry validates all three conditions and logs audit trail → Task M2

GIVEN	Student in JSS2A. Teacher assigned to Mathematics for JSS2A. Assessment config with CA1 (max 10). result_status is 'draft'.
WHEN	Three POST /api/scores/entry calls: (a) correct teacher, valid score 8, (b) correct teacher, score 15 (exceeds max 10), (c) different teacher not assigned to this class.
THEN	(a) Score saved. audit_logs row created with old_value=null and new_value={score:8}. (b) 400 returned — score exceeds max. No DB write. (c) 403 returned — teacher not assigned. No DB write.
✓ PASS IF	Case (a) saves and logs. Case (b) returns 400. Case (c) returns 403. audit_logs row exists for (a) only.
✗ FAIL IF	Case (b) saves an invalid score. Case (c) allows an unassigned teacher to write. audit_logs row missing for (a).
NOTE	Update case (a) with a new score (e.g. 9) and verify old_value={score:8} and new_value={score:9} in the updated audit log row.

TEST T11 Result computation returns correct totals, grades, positions, and tied position handling → Task M3

GIVEN	3 students in JSS2A with Mathematics scores entered. Weights: CA1 10%, CA2 10%, Mid-Term 10%, Exam 70%. Student A: 8,9,7,58 = 82 total. Student B: 10,9,10,64 = 93 total. Student C: 8,9,7,58 = 82 total (tied with A). Grading scale: A=70+, B=60-69.
WHEN	Call computeClassResults('JSS2A_id', 'term_id', 'school_id').
THEN	Student B: total=93, grade=A, position=1. Students A and C: total=82, grade=B, position=2 (tied). No student receives position=3 — it is skipped. computeStudentSubjectResult returns correct weighted total. getStudentsAtRisk

	returns empty (all above 40%).
✓ PASS IF	B is position 1. A and C are both position 2. Position 3 does not appear. Grades match grading scale.
✗ FAIL IF	Tied students receive different positions. Position 3 appears. Wrong grade assigned. Any DB write occurs (must be pure computation).
NOTE	<i>Verify no INSERT or UPDATE SQL runs during computation. The function must be side-effect free.</i>

TEST T12 Result workflow enforces valid transitions and blocks invalid ones → Task M4

GIVEN	JSS2A with Mathematics results. All students have complete scores. result_status='draft'.
WHEN	Four actions in sequence: (a) Teacher submits Mathematics for JSS2A. (b) Teacher attempts to submit again (already submitted). (c) Principal attempts to approve before all subjects are submitted. (d) Principal submits all subjects then approves.
THEN	(a) Status changes to 'submitted'. (b) 400 — already submitted. (c) 400 — not all subjects submitted. (d) Status changes to 'approved'. Each transition logged to audit_logs with user_id, old status, new status.
✓ PASS IF	All four responses match expected. audit_logs has entries for (a) and (d). Status never goes backwards without explicit return endpoint.
✗ FAIL IF	Any invalid transition succeeds. audit_logs entries missing. Direct jump from draft to approved possible.
NOTE	<i>Attempt POST /results/publish while status is 'submitted' (skipping approved). Must return 400.</i>

TEST T13 Report card PDF generates with correct school branding and student data → Task M5

GIVEN	School with logo uploaded, primary_colour set, name and motto in identity_config. Student with complete published results for current term. Teacher comments and principal remark saved.
WHEN	Call generateReportCard(studentId, termId, schoolId). Then GET /students/:studentId/report-card?term_id=.
THEN	PDF generated and uploaded to Supabase Storage at /report-cards/{schoolId}/{termId}/{studentId}.pdf. PDF URL returned by GET endpoint. PDF contains: school logo, name, motto, student name, results table with all subjects, grades, positions, teacher comment, principal remark, next term resumption date, promotion status.
✓ PASS IF	PDF URL returned. PDF opens and contains all required elements. School branding (logo, colour) visible.
✗ FAIL IF	PDF URL null or 404. PDF opens as blank or corrupted. School logo missing. Student name wrong.
NOTE	<i>Test with a student who has 15+ subjects to verify layout does not overflow the page boundary.</i>

TEST T14 Principal dashboard greets with title and returns correct school-wide stats → Task M6	
GIVEN	Principal user with title='Dr.' and name='Okafor' in the users table. School has students, published results, and submitted results.
WHEN	GET /dashboard/principal/overview using principal's JWT.
THEN	Response includes: greeting context resolves to 'Dr. Okafor' (verified in UI). total_students count correct. school_average computed from published results. Result status grid shows correct counts. Response time under 2 seconds. Second call within 5 minutes returns from cache.
✓ PASS IF	title='Dr.' reflected in greeting. All stat values match database counts. Cached response within 5 minutes.
✗ FAIL IF	title missing from greeting. Stats wrong. No cache (both calls hit DB with identical response times).
NOTE	Test cache by modifying a score between two calls within 5 minutes. The second call should still return the pre-change value (cache has not expired).

TEST T15 Teacher dashboard scopes data to assigned subjects and classes only → Task M7	
GIVEN	Teacher A assigned to Mathematics for JSS2A and JSS3A. Teacher B assigned to English for JSS2A.
WHEN	GET /dashboard/teacher/overview using Teacher A's JWT. Then GET /dashboard/teacher/score-entry-status.
THEN	Overview shows: my_subjects includes only Mathematics. my_classes includes only JSS2A and JSS3A. Score entry status shows JSS2A+Mathematics and JSS3A+Mathematics only. No English data visible. No JSS2B data visible.
✓ PASS IF	Teacher A sees only their own subjects and classes. Zero data from Teacher B's assignments visible.
✗ FAIL IF	Teacher A can see Teacher B's English scores or assignments. Any cross-assignment data leaks.
NOTE	Log in as Teacher B and verify Teacher B also sees only their own data. Cross-check that JSS2A appears for both but each sees different subjects.

TEST T16 Settings panel saves identity data and shows live colour preview → Task M8	
GIVEN	School admin logged in. Settings panel loaded at /settings/identity.
WHEN	Change primary_colour to #FF761B (Chronix orange). Upload a new school logo. Save.
THEN	PATCH /api/schools/:schoolId/identity called with new values. identity_config updated in school_settings. Logo uploaded to Supabase Storage. Portal header colour changes live as colour picker moves (before save). audit_logs entry created for the settings change.
✓ PASS IF	identity_config updated in DB. Logo URL updated. Live colour preview visible during picker interaction. audit_logs entry exists.

✗ FAIL IF	DB not updated after save. Logo URL unchanged. No live preview. No audit log.
NOTE	<i>Reload the page after saving and verify the new logo and colour persist from the database, not from local state.</i>

TEST T16b Settings locking returns 423 for assessment config with existing scores → Task M9	
GIVEN	Assessment config exists for current term. At least one score has been entered for this term.
WHEN	PATCH /api/schools/:schoolId/assessment-config/:configId with new weight values.
THEN	Response is 423 Locked with body: { success: false, error: { code: 'CONFIG_LOCKED', message: 'Assessment configuration cannot be changed after scores have been entered for this term.' } }. No database update occurs. audit_logs has no entry for this attempted change.
✓ PASS IF	423 returned. No DB change. Clear error code and message in response body.
✗ FAIL IF	Config updates despite scores existing. 200 returned. No 423 response.
NOTE	<i>Also test: PATCH grading scale when published results exist — must return 200 with warnings array, not 423. Two different lock behaviours for two different settings.</i>

TEST T17 Assessment config UI shows locked state and resolution preview → Task M10	
GIVEN	Two configs: Config A for current term with scores entered (locked). Config B for next term with no scores (editable).
WHEN	Open /settings/assessment-config. Click edit on Config A. Then click edit on Config B. Then use the resolution preview tool with a class and subject combination.
THEN	Config A: all inputs disabled, padlock icon visible, blue info banner shown. Config B: all inputs editable, weight total indicator live (green at 100, red otherwise). Resolution preview returns the correct config name and reason for the selected class+subject combination.
✓ PASS IF	Config A locked state renders correctly. Config B fully editable. Weight total indicator is real-time. Resolution preview returns correct config.
✗ FAIL IF	Config A inputs are editable. No padlock icon. Weight total not computed live. Resolution preview returns wrong config.
NOTE	<i>Set Config B weight to 99 and verify save button is disabled and total indicator is red.</i>

TEST T18 User creation sets teacher_mode and cannot be changed by the teacher → Task M11	
GIVEN	Admin creates two teacher accounts: Teacher A with teacher_mode='class', Teacher B with teacher_mode='subject'.
WHEN	Log in as Teacher A and attempt to change teacher_mode via profile settings. Also verify teacher_mode field visible to admin in user management.
THEN	teacher_mode appears in user management for admin. Teacher A's own profile page does not show or allow editing of teacher_mode. teacher_mode value persists

	correctly in the users table.
✓ PASS IF	teacher_mode visible to admin. Not visible or editable by teacher. Correct value stored in DB.
✗ FAIL IF	Teacher can see or edit their own teacher_mode. teacher_mode not stored in DB.
NOTE	<i>Attempt a direct API PATCH to /api/users/:userId with { teacher_mode: 'subject' } using Teacher A's JWT — must return 403.</i>

TEST T19 Score entry UI shows correct mode per teacher and submit is per class per subject → Task M12

GIVEN	Teacher A (class mode) assigned to Primary 3. Teacher B (subject mode) assigned to Mathematics for JSS2A and JSS3A.
WHEN	Log in as Teacher A and open score entry. Log in as Teacher B and open score entry. Teacher B enters scores for JSS2A and clicks submit.
THEN	Teacher A: class selector appears first, no assignment cards, no class tabs. Teacher B: assignment cards appear first, Mathematics card selected, JSS2A and JSS3A tabs shown, submit button reads 'Submit JSS 2A for approval'. Submitting JSS2A does not affect JSS3A status.
✓ PASS IF	Teacher A gets class-first flow. Teacher B gets subject-first flow. Submit button text is specific. JSS3A remains draft after JSS2A submitted.
✗ FAIL IF	Wrong mode shown. Submit button says generic text. Submitting one class changes another class status.
NOTE	<i>After Teacher B submits JSS2A, verify JSS3A tab still shows Draft status.</i>

TEST T20 Principal result approval uses dropdown selector and two-step publish → Task M13

GIVEN	Multiple classes with results in various states. Principal logged in at /principal/results.
WHEN	Select a class from the dropdown. Click Approve. Confirm in modal. Then click Publish. Read first confirmation. Click confirm. Read second confirmation. Click confirm again.
THEN	Dropdown shows all classes. Selecting a class loads its subject status grid. Approve button active only when all subjects are Submitted. First publish confirmation modal appears. Second publish confirmation modal appears after first. Results status changes to Published after both confirmations. Parent notification queued.
✓ PASS IF	Dropdown class selector works. Two confirmation modals appear in sequence. Status changes to Published only after both confirmations.
✗ FAIL IF	Horizontal scroll grid shown instead of dropdown. One confirmation modal or zero. Status changes on first click. Notification sent inline instead of queued.
NOTE	<i>Verify the notification queue table has a new row after publish — do not verify email was sent (that is tested in T27).</i>

PHASE 2 TESTS

Tests T21–T33 · Tasks O1–O13 · All must pass before launch

TEST T21 Attendance marks correctly, upserts on correction, and triggers 3-day alert → Task O1

GIVEN	Class JSS2A with 5 students. Teacher assigned as class teacher.
WHEN	Mark attendance for today: 4 present, 1 absent (Student X). Then mark same date again: change Student X to present (correction). Then mark Student X absent for 3 consecutive days total.
THEN	First marking: 5 attendance rows created. Correction: Student X row updated to present (not a new row). After 3 consecutive absents: attendance_alert row created for Student X. Parent notification queued.
✓ PASS IF	Upsert works — no duplicate rows on correction. Alert fires after 3 absents. Notification queued.
✗ FAIL IF	Duplicate rows created on correction. Alert does not fire. Alert fires after only 1 absent.
NOTE	Query: <code>SELECT COUNT(*) FROM attendance WHERE student_id=[X] AND date=[today]</code> — must return exactly 1 after correction.

TEST T22 Teacher attendance UI loads existing data for correction and exports CSV → Task O2

GIVEN	Attendance already marked for today for JSS2A. Teacher opens the attendance page.
WHEN	Navigate to /teacher/attendance with today's date and JSS2A selected. Change one student from Present to Absent. Save. Click Export CSV for the current month.
THEN	Existing attendance values pre-populate the toggle buttons (not all defaulting to Present). Changed student updates in DB. CSV downloads with correct columns: student name, admission number, each day of month, attendance percentage.
✓ PASS IF	Existing values pre-populated. DB updated after correction. CSV downloaded with correct structure.
✗ FAIL IF	All toggles default to Present despite existing data. DB not updated. CSV empty or malformed.
NOTE	Verify toggle buttons show correct existing state by inspecting the pre-loaded API response, not just visual appearance.

TEST T23 Parent portal shows correct child data and hides selector for single-child parent → Task O3

GIVEN	Parent A linked to one child (Fatima, JSS2A). Parent B linked to two children (Fatima JSS2A and Akin SS2A). Published results exist. Attendance marked.
WHEN	Log in as Parent A. Observe header. Log in as Parent B. Observe header. Switch children as Parent B.
THEN	Parent A: no child selector visible, Fatima's data shown directly. Parent B: child selector with chevron visible, shows Fatima's data by default. Switching to Akin loads

	Akin's data. Parent A cannot access Akin's data via any URL manipulation.
✓ PASS IF	Parent A has no selector. Parent B selector works. URL manipulation by Parent A returns 403 or empty for Akin's data.
✗ FAIL IF	Selector shown for single-child parent. Parent A can access other children's data. Switching children fails.
NOTE	Test URL manipulation: Parent A JWT + GET /students/[Akin_id]/results must return 403 or empty.

TEST T24 Student portal shows own results only and respects publication status → Task O4

GIVEN	Student Fatima with published results for Term 1 and draft results for Term 2.
WHEN	Log in as Fatima. View results page. Select Term 1. Select Term 2. Attempt to access another student's results via URL.
THEN	Term 1 results shown with all subjects, scores, grades. Term 2 shows 'Results not yet published — check back soon.' URL manipulation to another student's ID returns 403 or empty.
✓ PASS IF	Term 1 results visible. Term 2 shows pending message. Other student's data inaccessible.
✗ FAIL IF	Draft results visible to student. Another student's results accessible. No pending message for unpublished term.

TEST T25 Assignment submission blocked after due date — returns 400 → Task O5

GIVEN	Assignment created with due_date = yesterday. Student attempts submission.
WHEN	POST /api/assignments/:id/submit with a valid file attachment.
THEN	Response is 400 with body: { success: false, error: { code: 'DEADLINE_PASSED', message: 'The submission deadline for this assignment has passed.' } }. No submission row created in assignment_submissions.
✓ PASS IF	400 returned. No DB row created. Clear error message.
✗ FAIL IF	Submission succeeds after deadline. 200 returned. assignment_submissions row exists.
NOTE	Also test a submission exactly at the due_date minute — edge case should be tested explicitly.

TEST T26 Suspension-severity behaviour sends parent notification immediately, not queued → Task O6

GIVEN	Student with a linked parent who has a phone and email. Notification worker is running.
WHEN	POST /api/behaviour with severity='suspension' for the student.
THEN	behaviour_records row created. Parent notification sent immediately (not via the 30-second worker queue). notifications row exists for parent with type='behaviour_suspension'. notification_logs row shows status='sent'.

✓ PASS IF	Behaviour record created. Notification sent within 2 seconds. notification_logs shows 'sent' not 'queued'.
✗ FAIL IF	Notification sits in queue waiting for worker. Delayed by more than 30 seconds. notification_logs shows 'queued'.
NOTE	Compare: log a minor-severity incident and verify it goes through the queue. Log a suspension and verify immediate sending.

TEST T27 Notification worker delivers email via SendGrid and enforces role-pair messaging rules → Task O7

GIVEN	Parent A linked to Fatima (teacher: Mr Adebayo). Parent B not linked to Fatima.
WHEN	Worker running. (a) Parent A sends message to Mr Adebayo. (b) Parent B attempts to send message to Mr Adebayo (not their child's teacher).
THEN	(a) Message saved in messages table. Worker picks up notification and sends email via SendGrid to Mr Adebayo. (b) POST /messages returns 403 — recipient not valid for this sender.
✓ PASS IF	Case (a): message saved and email queued. Case (b): 403 returned, no message saved.
✗ FAIL IF	Case (b) succeeds — parent can message teachers of other children. SendGrid not called for case (a).
NOTE	Use SendGrid's activity feed or a test email address to verify email delivery. Do not mark pass based on API response alone.

TEST T28 Offline attendance saves to IndexedDB queue and syncs when reconnected → Task O8

GIVEN	Teacher logged in with class roster cached. Chrome DevTools Network set to Offline.
WHEN	Mark attendance for JSS2A while offline. Submit. Go back online.
THEN	While offline: success toast shown 'Attendance saved locally — will sync when reconnected.' Header shows red offline indicator with pending count. attendance rows NOT in database yet. After reconnecting: auto-sync fires, attendance rows appear in database, header shows green indicator, pending count clears.
✓ PASS IF	Offline toast shown. DB empty while offline. DB populated after reconnect. Header indicator correct in all three states.
✗ FAIL IF	DB write attempted while offline (fails silently). No toast shown. Data lost after reconnect. Header indicator static.
NOTE	Verify IndexedDB contents in Chrome DevTools > Application > IndexedDB > offline_attendance_queue before and after sync.

TEST T29 Notification settings saves event toggles and displays SMS sender field as disabled → Task O9

GIVEN	Admin logged in. /settings/notifications loaded.
WHEN	Toggle 'Student absent' SMS notification to OFF. Save. Reload page. Observe SMS sender name field.

THEN	Toggle state persists after reload (stored in notification_config JSONB). SMS sender name field visible but greyed out with label 'Available in Phase 3'. Email test button sends test email to admin's email address.
✓ PASS IF	Toggle persists after reload. SMS field visible but disabled. Email test button works.
✗ FAIL IF	Toggle resets after reload. SMS field editable. Email test button absent or broken.
NOTE	Verify toggle persists by querying: <code>SELECT notification_config FROM school_settings WHERE school_id=[id]</code> and checking the JSON directly.

TEST T30 Registrar promotion manager creates new student_classes row without deleting old one → Task O10

GIVEN	Student Fatima in JSS2A for session 2024/2025. New session 2025/2026 created. End-of-session promotion manager open.
WHEN	Mark Fatima as Promoted. Click Bulk Promote. Confirm.
THEN	New student_classes row created for Fatima in JSS3A for session 2025/2026. Old student_classes row for JSS2A in 2024/2025 still exists. GET /students/:id returns both class history entries. Fatima's report card for Term 1 2024/2025 still generates correctly using old class data.
✓ PASS IF	Two student_classes rows exist for Fatima (one per session). Old row preserved. Historical report card generates.
✗ FAIL IF	Old student_classes row deleted. Only one row exists. Historical report card fails due to missing class data.
NOTE	Attempt to generate Fatima's report card for the previous session after promotion. It must still work.

TEST T31 All 5 Phase 2 integration tests pass → Task O11

GIVEN	Phase 2 fully built and individually tested.
WHEN	Run the Phase 2 integration test suite: <code>npm run test:integration</code> in /apps/api.
THEN	All 5 tests pass: offline sync test, parent data isolation test, notification pipeline test with SendGrid mock, behaviour auto-alert test, assignment deadline enforcement test. Zero failures. Zero skips.
✓ PASS IF	All 5 tests pass with green output.
✗ FAIL IF	Any test fails. Any test skipped. Test suite does not run.
NOTE	Do not proceed to Phase 3 or production deployment until this test suite passes completely.

TEST T32 Child Prime Onyx School data imported — all students, teachers, and classes exist → Task O12

GIVEN	Student list, class list, teacher list, and staff titles collected from Child Prime Onyx School.
-------	--

WHEN	Run data import script. Query the database for counts.
THEN	All students registered with correct names, classes, and admission numbers. All teachers created with correct title (Mr/Mrs/Dr/etc.), subject assignments, and teacher_mode. All classes created. Academic session 2025/2026 First Term created and set as active.
✓ PASS IF	Student count matches school register. Teacher count matches staff list. All classes exist. Active session is First Term 2025/2026.
✗ FAIL IF	Any student missing. Any teacher missing or missing title. Wrong session active.
NOTE	<i>This is a manual task, not a code task. Verify by printing a student list from the system and comparing to the school's paper register.</i>

TEST T33 Production deployment live at edu.chronixtechnology.com with valid SSL → Task O13	
GIVEN	Vercel project configured with Next.js app. Railway project configured with Express API. All environment variables set in both platforms. Domain edu.chronixtechnology.com pointing to Vercel.
WHEN	Visit https://edu.chronixtechnology.com in a browser. Log in as principal. Navigate to dashboard.
THEN	Site loads over HTTPS with valid SSL certificate. Login works. Principal dashboard loads with correct school data. No environment variable errors in Railway logs. API health check endpoint returns 200.
✓ PASS IF	Site loads on HTTPS. SSL valid. Login succeeds. Dashboard loads. No 500 errors in logs.
✗ FAIL IF	HTTP redirect not working. SSL certificate invalid or expired. Login fails. Dashboard returns errors.
NOTE	<i>Run the startup env var validation check in Railway logs — it must show all variables present, not a startup error.</i>

PHASE 3 STRETCH TESTS

Tests T34–T42 · Tasks S1–S9 · Run only after Phase 2 is live

TEST T34 Fee invoice generation creates correct invoices for all students → Task S1

GIVEN	School with fee structures: school-wide ₦50,000 tuition and JSS2A-specific ₦10,000 lab fee. Both for current term.
WHEN	POST /finance/invoices/generate with current term_id.
THEN	All students get invoice with total_amount=₦50,000 (tuition only) EXCEPT JSS2A students who get ₦60,000 (tuition + lab fee). Running generate twice does not create duplicate invoices (upsert). fee_invoices rows exist for every student.
✓ PASS IF	Correct total per student. No duplicates on second run. All students have invoices.
✗ FAIL IF	Wrong total for any student. Duplicate invoices created. Any student without invoice.

TEST T35 Paystack Pay Now button activates and verifies payment server-side → Task S2

GIVEN	Student invoice with ₦45,000 outstanding. Parent portal loaded with fee card visible.
WHEN	Click Pay Now. Complete Paystack payment flow with test card. Return to parent portal.
THEN	Paystack initiate endpoint called with correct amount in kobo (4500000). Parent redirected to Paystack. After payment: callback endpoint verifies transaction via Paystack API (not just callback params). payments row created with method='paystack'. Invoice balance updated to ₦0. Invoice status='paid'. Pay Now button replaced with 'Paid in full' badge.
✓ PASS IF	Payment verified server-side. payments row created. Invoice balance correct. UI reflects paid status.
✗ FAIL IF	Payment recorded without server-side verification. Callback params trusted directly. Balance not updated.
NOTE	Test with Paystack test card 4084084084084081. Verify transaction on Paystack dashboard.

TEST T36 Bursar dashboard generates PDF receipt with correct details → Task S3

GIVEN	A manual cash payment of ₦25,000 recorded for Student Fatima by the bursar.
WHEN	Bursar clicks Print Receipt for Fatima's payment.
THEN	PDF downloads containing: school name and logo, student name and admission number, amount paid (₦25,000), payment date, payment method (Cash), reference number, remaining balance, school stamp image.
✓ PASS IF	PDF downloads. All listed fields present and correct.
✗ FAIL IF	PDF blank or corrupted. Any field missing. Wrong amount shown.

TEST T37 Analytics snapshots generated nightly and returned with trend comparison → Task S4	
GIVEN	School with 2 terms of published results. Analytics service configured. node-cron job scheduled at 2am.
WHEN	Manually trigger POST /analytics/refresh. Then GET /analytics/overview.
THEN	school_analytics_snapshots rows created for current term. Overview response includes current term stats AND previous term stats for comparison. Trend arrows computable from data. Response time under 500ms (reading from snapshot, not computing live).
✓ PASS IF	Snapshot rows created. Overview includes both terms. Response under 500ms.
✗ FAIL IF	Live computation on every request. No snapshot rows. Previous term data missing.
NOTE	Query school_analytics_snapshots directly to verify rows exist before testing the overview endpoint.

TEST T38 Timetable rejects teacher and class clashes with 409 → Task S5	
GIVEN	Teacher Mr Adebayo assigned to Mathematics JSS2A on Monday Period 1.
WHEN	Attempt to add: (a) English JSS2A on Monday Period 1 (class clash). (b) Further Maths JSS3A on Monday Period 1 (teacher clash — same teacher, same time).
THEN	(a) 409 returned with message: 'JSS 2A already has a class scheduled on Monday Period 1 (Mathematics).' (b) 409 returned with message: 'Mr Adebayo is already teaching JSS 2A Mathematics on Monday Period 1.' Neither slot created in DB.
✓ PASS IF	Both cases return 409 with specific clash descriptions. No DB rows created for either.
✗ FAIL IF	Either clash accepted silently. 409 without description. DB rows created despite clash.

TEST T39 Termii SMS sends with school-specific sender name and respects daily throttle → Task S6	
GIVEN	School with Termii API key stored in school_settings. SMS sender name set to 'CPOXYS' (Child Prime Onyx School). Parent with Nigerian phone number.
WHEN	Trigger 4 SMS notifications to the same parent in one day.
THEN	First 3 SMS delivered via Termii API with from='CPOXYS'. 4th SMS attempt blocked by throttle (3/day limit). notification_logs shows 3 'sent' and 1 'throttled' entries.
✓ PASS IF	Sender name correct. First 3 sent. 4th blocked. notification_logs accurate.
✗ FAIL IF	Sender name wrong or missing. 4th SMS sent. No throttle. notification_logs missing entries.
NOTE	Verify Termii API key comes from school_settings, not from environment variable. Check with SELECT notification_config FROM school_settings.

TEST T40 Fee reminder cron sends to outstanding parents on Monday 8am → Task

S7	
GIVEN	3 students: Student A (paid), Student B (partial), Student C (unpaid). Monday morning. Cron job enabled.
WHEN	Wait for Monday 8am OR manually trigger the fee reminder endpoint.
THEN	Student A's parent receives no reminder. Student B's and C's parents receive email reminders via SendGrid. notification_logs shows 2 'sent' entries. Bursar's manual trigger button produces identical results.
✓ PASS IF	Only outstanding parents notified. Paid parent not notified. notification_logs accurate.
✗ FAIL IF	All parents notified including paid. notification_logs missing. Manual trigger fails.

TEST T41 Report card settings live preview uses dummy data and real branding → Task S8	
GIVEN	School with logo uploaded, primary_colour set, and principal signature uploaded. Admin on /settings/report-card.
WHEN	Click Live Preview button.
THEN	PDF generates within 10 seconds. PDF opens in new tab. PDF shows: real school logo, real school name, real primary_colour in table headers, real principal signature, dummy student name (not a real student). No real student data appears anywhere in the preview.
✓ PASS IF	PDF generated with real branding. Dummy student data used. No real student data in preview.
✗ FAIL IF	Real student data used. PDF blank. PDF uses default branding instead of school branding.
NOTE	<i>Query students table and verify the student name in the preview PDF does not match any real student.</i>

TEST T42 Production Phase 3 readiness — all endpoints respond under 2000ms at 50 concurrent users → Task S9	
GIVEN	Phase 3 fully built and deployed to Railway production environment.
WHEN	Run Artillery load test: artillery run chronixedu-load-test.yml with config: 50 virtual users, 60 second duration, targeting the 5 critical endpoints (class scores, student report card, attendance by class+date, outstanding invoices, analytics overview).
THEN	All endpoints respond under 2000ms at p95. Zero 500 errors. Zero 503 errors. Error rate under 0.1%. Artillery HTML report generated and saved.
✓ PASS IF	All endpoints under 2000ms at p95. Zero 5xx errors.
✗ FAIL IF	Any endpoint over 2000ms at p95. Any 500 or 503 errors. Error rate above 0.1%.
NOTE	<i>If any endpoint fails: run EXPLAIN ANALYZE on its query and add missing indexes before retrying the load test.</i>

Test Index — All 42 Tests

Test	Task	What It Proves	Phase	Result
T1	F1	Both apps start without errors	Foundation	—
T2	F2	16 tables + title + teacher_mode created correctly	Foundation	—
T3	F3	School A cannot see School B data — CRITICAL	Foundation	—
T4	F4	JWT returns correct claims including title	Foundation	—
T5	F5	School creation seeds Nigerian defaults	Foundation	—
T6	F6	Current-context cached, single-active constraint enforced	Foundation	—
T7	F7	Teacher mode locked per account, no toggle in UI	Foundation	—
T8	F8	4-level assessment config resolution correct	Foundation	—
T9	M1	Student registration creates all 5 rows	Phase 1	—
T10	M2	Score entry validates 3 conditions and logs audit	Phase 1	—
T11	M3	Computation correct — totals, grades, tied positions	Phase 1	—
T12	M4	Status transitions enforced — no invalid jumps	Phase 1	—
T13	M5	PDF generated with branding and student data	Phase 1	—
T14	M6	Principal greeting uses title, stats cached	Phase 1	—
T15	M7	Teacher dashboard scoped to own assignments only	Phase 1	—
T16	M8	Settings saves and live colour preview works	Phase 1	—
T16b	M9	423 on config change with existing scores	Phase 1	—
T17	M10	Locked config UI correct, resolution preview accurate	Phase 1	—
T18	M11	teacher_mode set by admin, not editable by teacher	Phase 1	—
T19	M12	Subject mode and class mode render correctly per teacher	Phase 1	—
T20	M13	Dropdown selector, two-step publish, notification queued	Phase 1	—
T21	O1	Attendance upserts, 3-day alert fires	Phase 2	—
T22	O2	Existing data pre-populated, CSV exports correctly	Phase 2	—
T23	O3	Single-child no selector, cross-child access blocked	Phase 2	—
T24	O4	Student sees own data only, draft results hidden	Phase 2	—
T25	O5	Post-deadline submission blocked with 400	Phase 2	—
T26	O6	Suspension notification immediate, not queued	Phase 2	—
T27	O7	SendGrid email delivered, role-pair rules enforced	Phase 2	—
T28	O8	Offline queue saves, syncs on reconnect	Phase 2	—
T29	O9	Toggles persist, SMS field disabled at launch	Phase 2	—
T30	O10	Promotion creates new row, preserves old row	Phase 2	—
T31	O11	All 5 integration tests pass	Phase 2	—
T32	O12	All school data imported correctly	Phase 2	—
T33	O13	Production live at edu.chronixtechnology.com with SSL	Phase 2	—
T34	S1	Fee invoices correct per class, no duplicates	Phase 3	—

T35	S2	Paystack server-side verified, balance updated	Phase 3	—
T36	S3	PDF receipt contains all required fields	Phase 3	—
T37	S4	Analytics snapshot generated, trend data returned	Phase 3	—
T38	S5	Timetable clash returns 409 with description	Phase 3	—
T39	S6	Termii uses school sender name, throttle enforced	Phase 3	—
T40	S7	Fee reminders sent to outstanding parents only	Phase 3	—
T41	S8	Live preview uses real branding, no real student data	Phase 3	—
T42	S9	All endpoints under 2000ms at 50 concurrent users	Phase 3	—